

告别头文件! C++ modules实战

01

模块与头文件

- 头文件的缺点
- 模块的优势
- workshop1,2

02

模块语法简介

- Export Declaration
- Module Declaration
- Import Declaration
- Module的基本代码结构
- Header Unit
- Workshop 3, 4

03

模块化工程

- 模块工程结构的变化
- 兼容历史代码：Module Wrapper
- 阿里云在hologres中的模块化改造
- Workshop 5

01 模块(Modules)与头文件

Hello world, C++ modules!

传统的头文件写法:

```
#include <iostream>
int main() {
    std::cout<<"Hello world!"<<std::endl;
}
```

现代化的模块写法:

```
import std;
int main() {
    std::cout<<"Hello world!"<<std::endl;
}
```


头文件的痛点



大量的重复编译

C++用户常常抱怨C++编译速度慢，其中一个原因就来自于头文件。



不够“卫生”，被其他代码影响

头文件采用简单的预处理方式，简单粗暴的“复制黏贴”到代码中，这种做法是不够卫生的，头文件中的内容非常容易被其他头文件干扰。



缺乏封装

头文件将所有的声明都暴露给了用户，无法将某些内部实现隐藏起来。



不一致的声明

在头文件架构中，各个翻译单元 (*.cpp)之间都是独立的，编译器无法检查出现在不同翻译单元中的不一致的声明。

C++ Modules: 不止于import

模块是如何解决头文件的痛点的？



模块的编译提速

模块其本身可以被编译
当我们导入一个模块时，不需要像头文件那样再次
编译代码



模块不会被外部代码影响

模块是“卫生”的，由于它被独立编译，因此不会
像头文件那样被之前的其他代码影响



模块的封装性

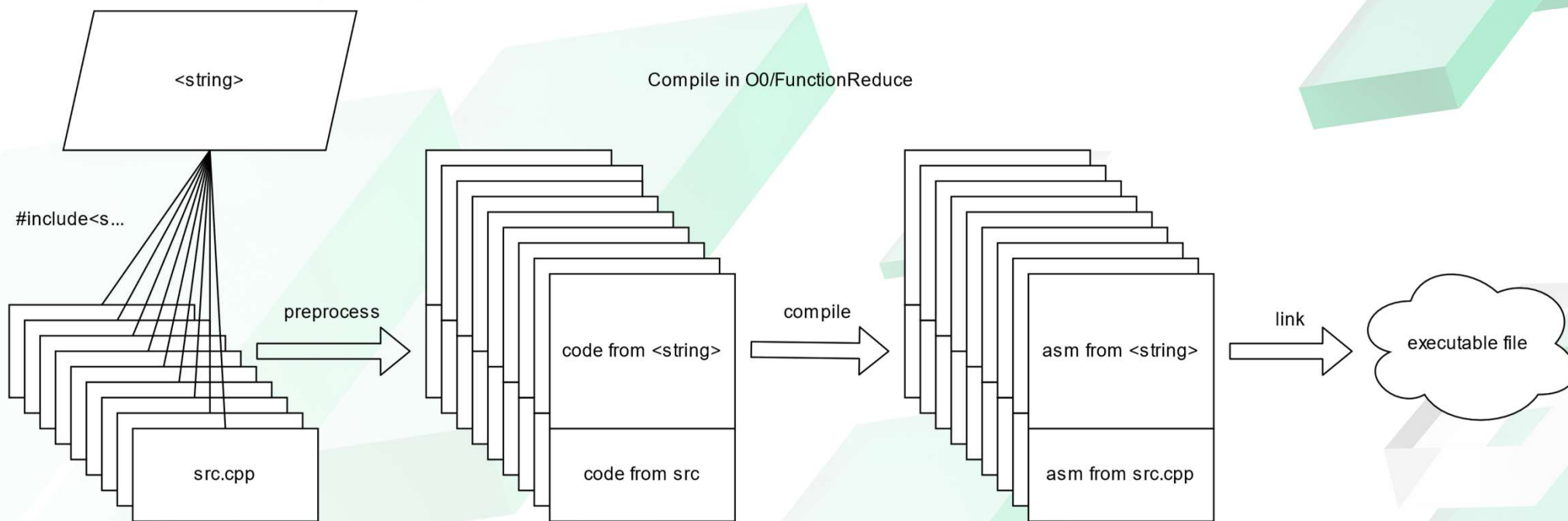
在C++模块中，我们可以通过**export**关键字控
制哪些声明会被暴露给用户



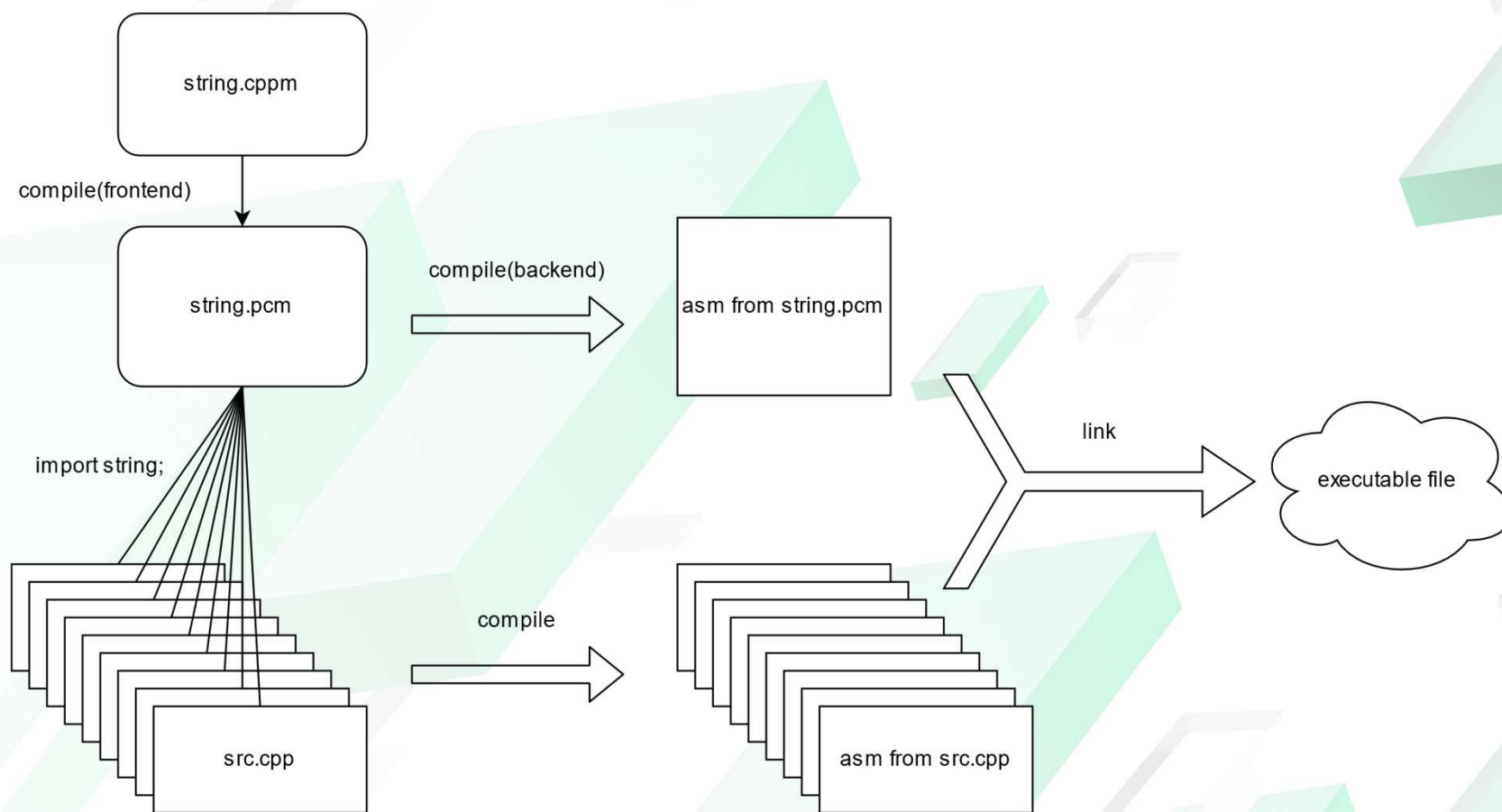
检查不一致的声明

导入模块时，可以检查不同模块之间是否出现了多
重定义，加强了编译器检查错误的能力。

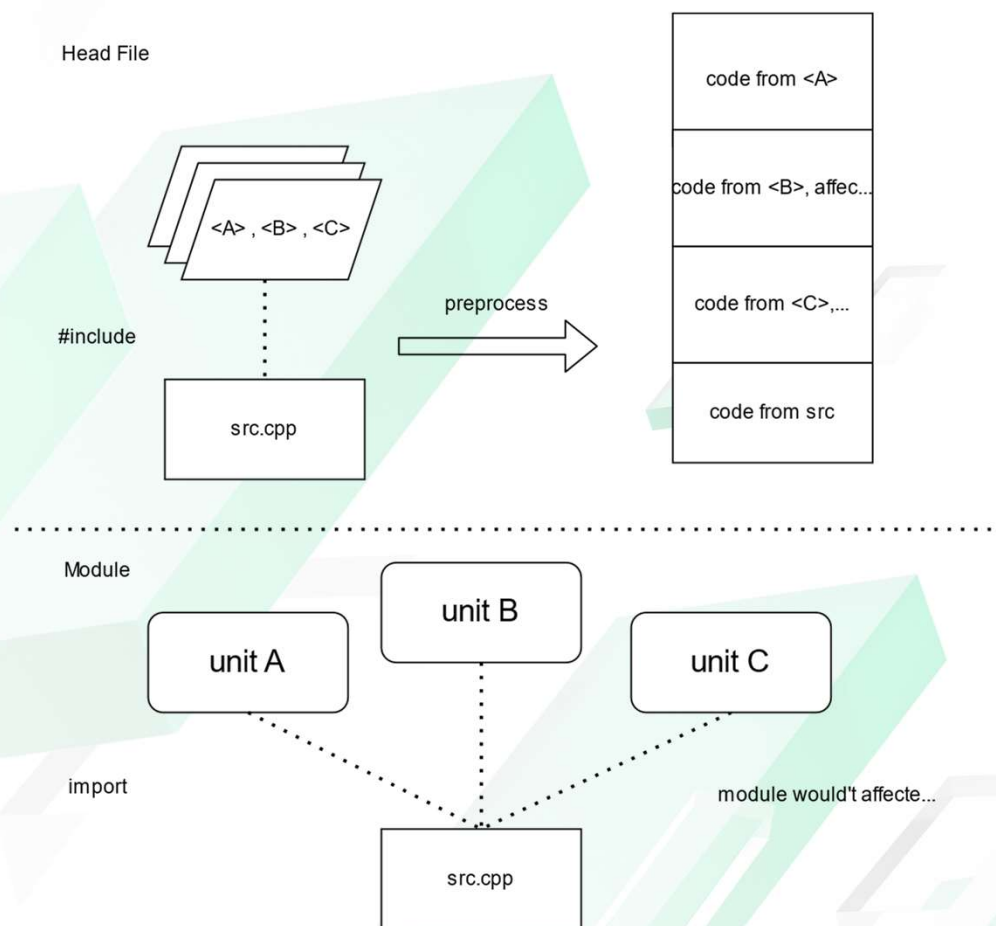
痛点：重复编译



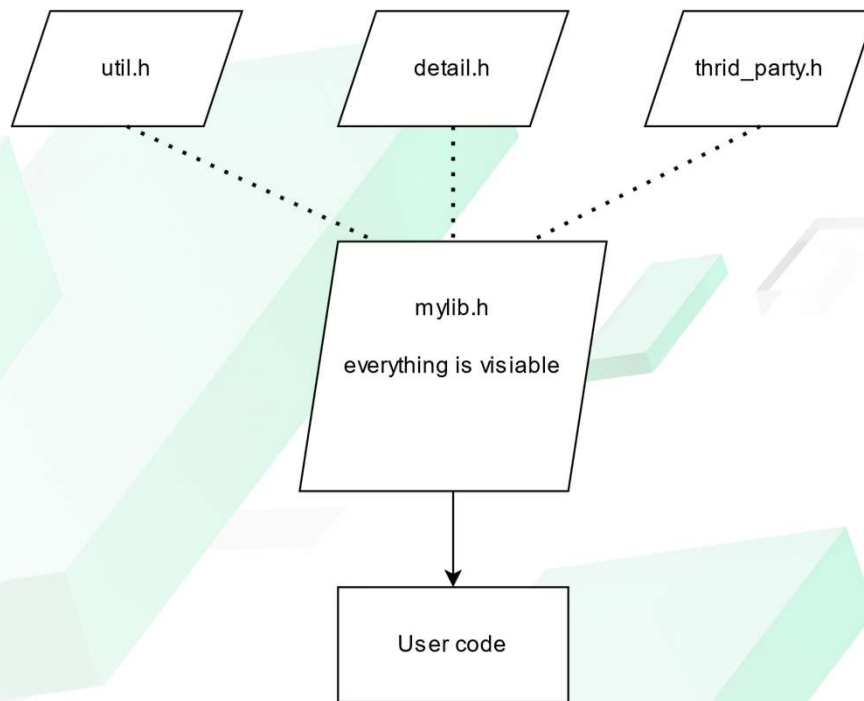
模块的编译提速



头文件易被其他代码影响，而模块不会



头文件中所有的声明都会暴露给外部



模块通过使用export 关键字标明哪些声明会导出给外部

WorkShop环境介绍

本次我们会使用在线环境完成编程任务，学习如何使用module编程。

在线环境已经预装好了基本的开发环境和代码，您只需使用ssh连接服务器，即可开始编程。

ssh workshop@47.238.35.119 -p 500xx

密码：workshop

端口号：50001-50099，大家可以任一选择一个端口连接，先到先得。

登上服务器后，请大家修改自己的密码：su workshop 防止别人误登录。

并在直播上打出自己选择的端口号（1-99）

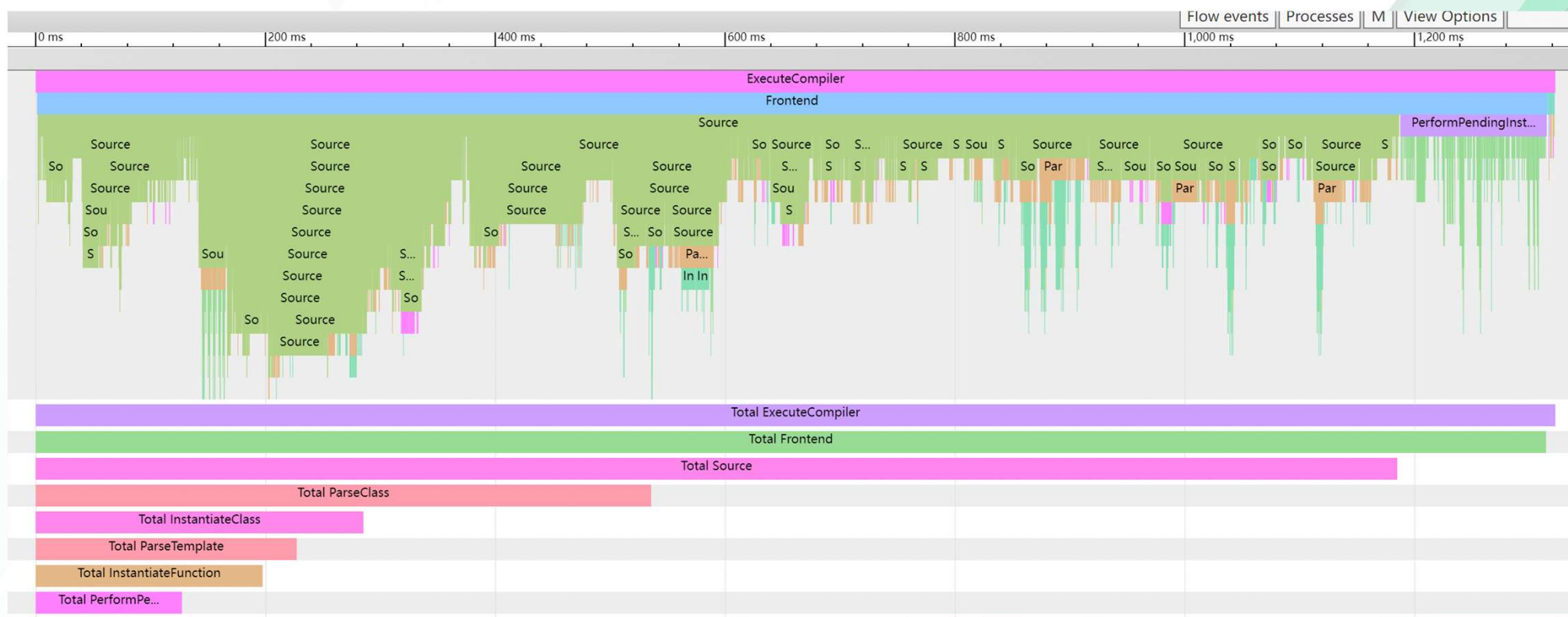
本次活动一共有5个workshop任务，我们将会边讲解module边完成这些任务。

自己搭建环境（使用docker或手动）：<https://github.com/poor-circle/workshop>

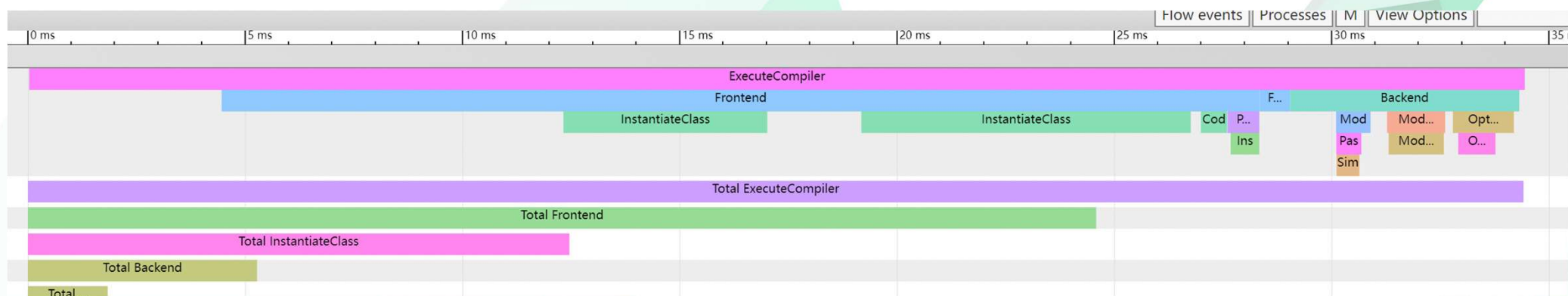
WorkShop 1: GoodBye Head File!

WorkShop 2: Hello world, C++ modules!

使用头文件的编译时间(1.3s)



使用模块的编译时间(30ms)



02 模块语法简介

Export Declaration

模块回收了关键字export，用于表示该声明会被导出给外部代码使用。

```
export inline int a; // 导出一个变量

export void foo(); // 导出一个函数声明

export void foo(){...} // 导出一个函数实现

export class A {}; // 导出一个类

export enum B {}; // 导出一个枚举

export namespace my_lib {}; // 导出一个名称空间

export template<typename T> C{}; // 导出模板

export using std::max; // 导出using declaration

export using D=std::vector<int>; // 导出别名
```

Export Declaration

有些东西无法被export:

1. 宏: 宏只存在于预处理阶段, 因此一个模块无法导出一个宏。同样的, 外部代码中的宏也不会影响模块内部中的代码。
2. using namespace

```
export using namespace std; // 不合法!
```

这并不是一个好主意, 很容易破坏外部代码的预期行为, C++模块干脆禁止了这样的声明。

Module Declaration

一个模块可以由多个模块单元 (Unit) 组成，每个模块单元对应了一份代码文件，文件通常会使用一个特殊的后缀名(cppm, ixx)。

在一个模块单元中，有且只有一个模块声明 (Module Declaration)，表示这个单元从属于哪一个模块。

```
module Foo; //声明一个名为Foo的模块
```

```
export module Foo.Bar; //声明一个名为Foo.Bar的模块
```

```
module Foo.Bar.Gua; //声明一个名为Foo.Bar.Gua的模块
```

需要注意的是，模块名字中的.符号，没有任何特殊的语义，不保证Foo.Bar和Foo之间有从属关系。（需要用户自己组织）

Module Declaration

根据Module Declaration的不同，我们可以将模块单元区分成：

1. 接口单元 (interface unit)

一个模块只能有一个接口单元，只能在这个单元中使用export declaration对外暴露接口。

2. 实现单元 (implementation unit)

一个模块可以有多个实现单元，实现单元里不能使用export declaration

```
// Foo.cppm
export module Foo; // interface unit
// Foo_impl1.cpp
module Foo; // implementation unit
// Foo_impl2.cpp
module Foo; // implementation unit
// Foo_impl3.cpp
module Foo; // implementation unit
```

Module Declaration

通常来说，在模块中，用户不需要像传统的头文件工程那样，将声明与实现分离开来。

然而，依然存在一些特殊情况，例如，对于汇编，我们可能希望分离声明与实现，并通过构建工具在不同平台上选择不同的实现。

我们依然可以通过Implementation Unit实现这一点：

```
export module thread; //Interface Unit
class thread_context;
void switch_in(thread_context* to);
void switch_out(thread_context* from);
```

```
module thread; //Implementation Unit
class thread_context;
{
    //define something
}
void switch_in(thread_context* to)
{
    //do something
}
void switch_out(thread_context* from)
{
    //do something
}
```


Module Declaration

我们可以将一个模块拆成多个分区，分区不是一个独立的模块，它不能被单独使用。

```
export module Foo.Bar:part1; //声明属于Foo.Bar模块的一个分区 (Partition)
```

```
export module Foo.bar:part1:part2; // 不合法! 分区不能嵌套!
```

```
module Foo.Bar:part1; //分区也具有Interface Unit和Implement Unit的区别
```

Import Declaration

我们可以通过Import Declaration导入其他的模块。

```
// main.cpp
import std;
import foo;
import foo.bar:part1;
// foo.cppm
export import foo.bar;
import std;
// foo.bar.cppm
export import :part1;
```

// 导入标准库模块
// 导入foo.bar模块
// 不合法! 不能导入一个其他模块的分区模块
// 导入foo.bar模块, 并将其再导出给外部用户
// 导入std模块, 但是不暴露给外部用户
// 导入自己的分区模块part1, 并暴露给外部用户

模块代码的基本结构

```
/*-----Global Module Fragment -----*/  
module;  
#include“util.hpp” //在模块代码中包含头文件  
/*-----Module Declaration-----*/  
export module http.client; // 在模块代码中包含头文件  
/*-----Import Declaration-----*/  
import std; // 导入其他模块  
import asio;  
export import cppjson;  
import openssl;  
/*-----用户代码-----*/  
namespace http  
{  
    namespace detail  
    {  
        class helper // 无export前缀, 不会暴露给外部用户  
        {  
            //.....  
        }  
    }  
    // Export Declaration  
    export enum class status  
    {  
        OK,  
        NotFound,  
        //.....  
    }  
    export class client  
    {  
        tcp::socket soc;  
        //.....  
    };  
    export int foo();  
}
```

Header Unit

和之前提到的模块 (Named Modules) 不同, Header Unit是另外一种模块, 其形式完全不同。

之前的提到的模块语法主要来自于微软在2014年的提案, 然而, Google认为这一设计无法导出宏, 是一个致命缺陷, 为此, Google提出了自己的方案: Header Unit。

处于纯粹的兼容性目的, Header unit的效果和#include头文件完全相同 (代码会引入宏, 会被宏影响)。然而, 这使得编译器只能对header unit做有限度的预处理, 从而大大限制了编译器的加速空间。

```
import <iostream>;  
//和#include <iostream>的效果完全相同。  
//会引入宏, iostream中的代码也可能被宏影响  
int main()  
{  
    std::cout<<"Hello world"<<std::endl;  
}
```

总结：模块单元的分类



Interface Unit

一个模块只能有一个Interface，在这里声明哪些符号暴露给外部用户



Implementation Unit

一个模块可以有多个Implementation Unit，可以这里实现代码。



Partition Unit

一个模块可以划分为多个分区模块，分区模块不是独立是，是主模块的一部分



Header Unit

最特殊的一种模块单元。它直接从头文件转换得来。

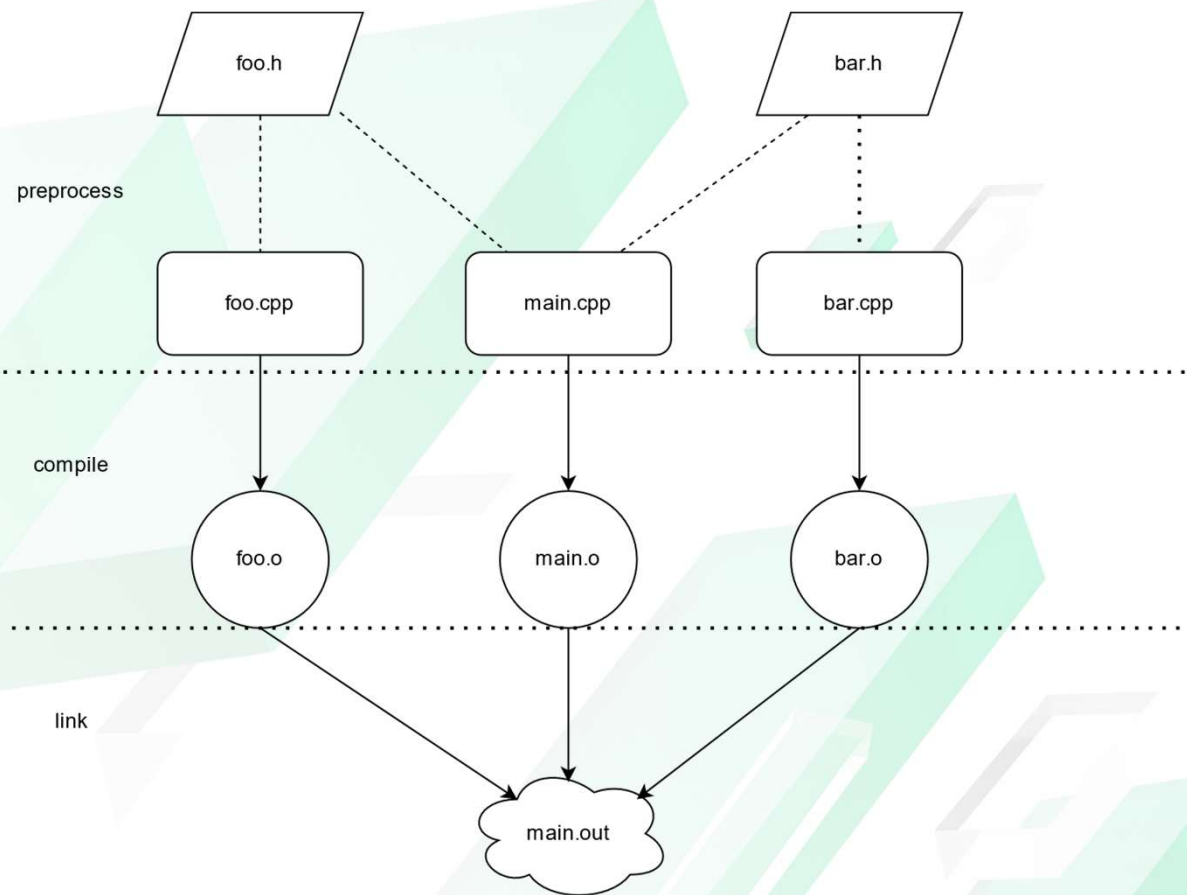
WorkShop 3: 编写我们的模块

WorkShop 4: 多模块单元

03 模块化工程

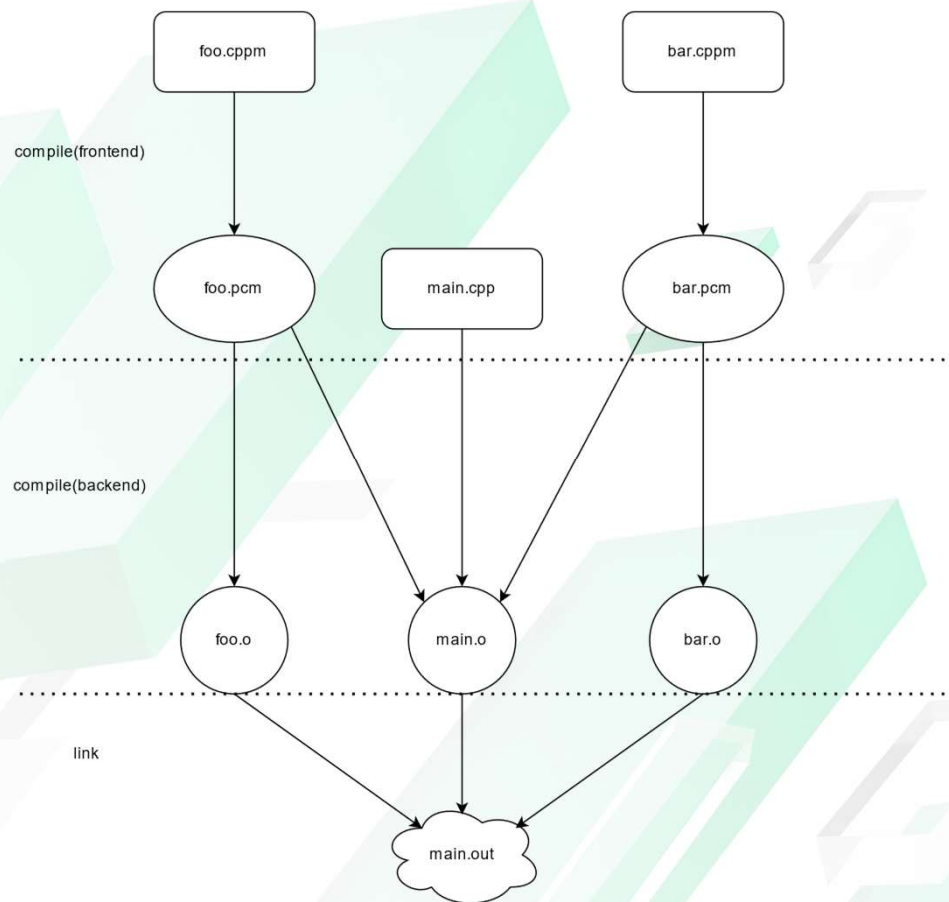
传统C++工程架构

各个翻译单元(*.cpp)在编译时完全独立，互不影响。头文件被多次重复包含并编译



基于模块的C++工程架构

代码存在依赖关系，一个翻译单元需要依赖其他翻译单元的编译结果



模块化工程带来的变化

项目组织

模块工程具有和传统的头文件工程不同的组织架构

依赖分析

构建工具需要能够正确的分析各个模块之间的依赖关系，得到正确的构建顺序。

兼容历史代码：Module Wrapper

一方面，C++具有大量的基于头文件的历史代码，我们希望以尽可能小的代价将其模块化。

另外一方面，我们也希望改造后的代码能够同时兼容头文件和模块两种架构模式，方便平滑过渡。

Module Wrapper

通过export using declaration, 我们可以快速将已有的头文件封装到模块中, 不影响旧代码。

```
// iostream.cppm
module;
#include<iostream>
export module iostream;
namespace std {
    export using std::cin;
    export using std::cout;
    export using std::endl;
}
// main.cpp
import iostream;
int main() {
    std::cout<<"Hello Module Wrapper"<<std::endl;
}
```


Module Wrapper

通过export extern c++和模块控制宏，我们可以一口气导出所有的符号，并兼容头文件和模块两种写法

```
// hello.hpp
#ifdef HELLO_USE_MODULE
import std;
#else
#include<iostream>
#include<vector>
#endif

void hello() {
    // ...
}

// hello.cppm
module;
export module hello;
export extern "C++" {
    #define HELLO_USE_MODULE
    #include<hello.hpp>
}
```

WorkShop 5: 将传统的头文件项目转换为模块

模块化重构辅助工具

模块化重构的过程中存在许多机械的工作，我们实验性的提供了一些自动化工具用于减少模块重构中的工作量。相关工具可能会在未来合入Clang主线

它能做到：

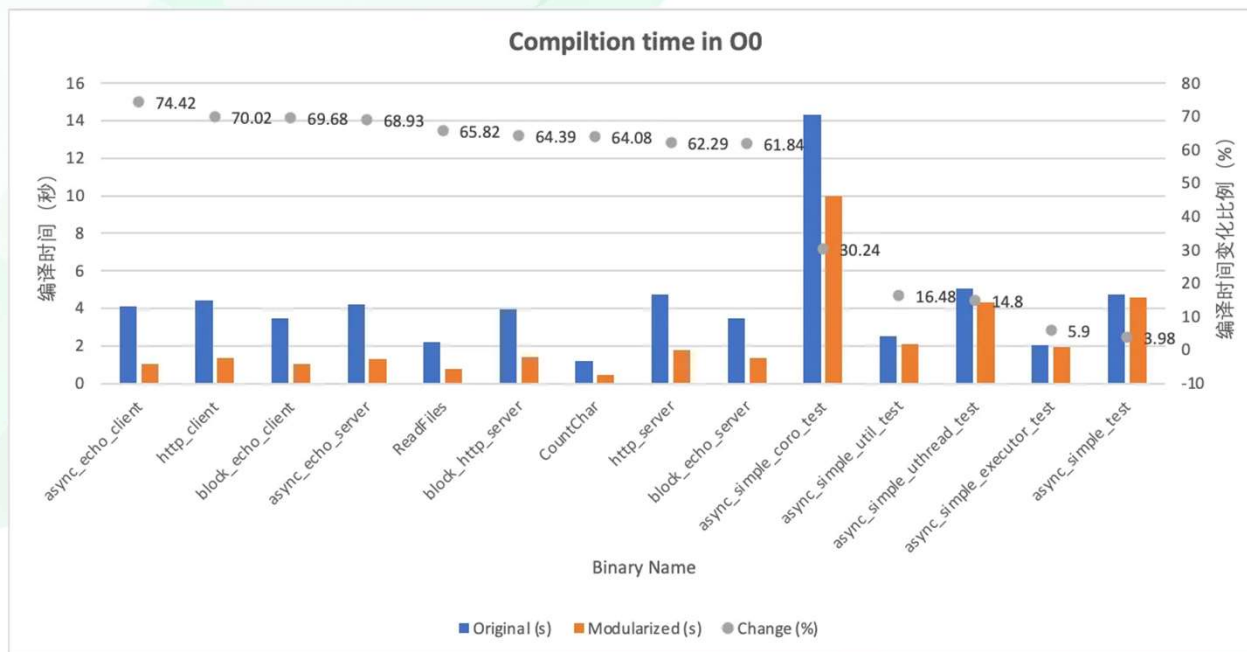
1. 自动插入宏，控制项目使用头文件还是模块
2. 自动扫描头文件之间的依赖关系（虽然不太准确）
3. 自动生成module wrapper文件。

详见：<https://github.com/ChuanqiXu9/clang-modules-converter>

开源项目中的模块化改造- async_simple

async_simple是阿里巴巴提供的异步组件库，支持C++20无栈协程，C++有栈协程并提供了许多异步工具。async_simple在2021年末就开始使用module进行改造，并取得了较好的编译速度提升。

https://github.com/alibaba/async_simple



商业项目中的模块化改造- Hologres

阿里云计算平台 Hologres 和 Alibaba Cloud Compiler 的开发团队积极合作，自 2022 年起便开始尝试使用 Modules。如今 Hologres 已将 Modules 作为默认开发模式并上线了基于 Modules 编译的镜像，至今已稳定运行 5 个月份。

Hologres “可能”是世界上首个大规模使用 Modules 的商业项目。



编译加速

User time下降了42%，Real time下降25%



减少构建产物的体积

动态库的体积下降了 12%



更强的编译期ODR检测

在编译期可以直接检测到不同模块中的多重声明而报错

The background features a series of concentric rings of 3D rectangular blocks. The blocks are colored in alternating shades of light green and white, creating a radial pattern that draws the eye towards the center. The perspective is from above, looking down into the center of the rings.

Q & A